# Introduction to Optimization

## CO 250

Kevin Shu

Lecture 2

Live (interactive) version of these slides: https://ootks.github.io/co250-lectures/lecture2.html

# Lecture Outline

- Linear programming review
- Integer (Linear) Programs

# Linear Programming Review

# Linear Programming Review

A linear program is an optimization problem of the form

$$\begin{array}{ll} \min & c^\intercal x \\ \text{such that} & Ax \leq b \end{array}$$

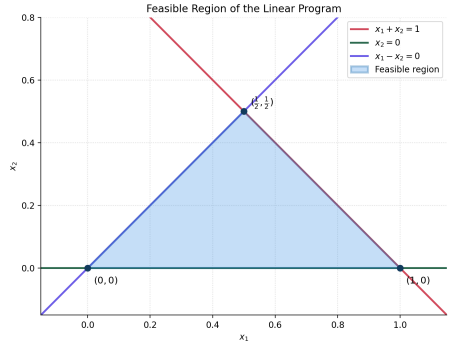
$A$ is an $m \times n$ matrix, $b \in \mathbb{R}^m$, $c, x \in \mathbb{R}^n$.

The min can also be a max, and we can mix $\leq$ and $\geq$ in the constraints. These changes can be made by flipping signs.

# Linear Programming Review

The following is an LP

$$\begin{array}{ll} \max & x_2 \\ \text{such that} & x_1 + x_2 \leq 1 \\ & x_1 - x_2 \geq 0 \\ & x_2 \geq 0. \end{array}$$



# Diet Problem

**Nutrition per standard serving**

| Nutrient | Egg | Banana | Chicken Breast |
|---|---|---|---|
| Calories | 72 kcal | 105 kcal | 128 kcal |
| Protein | 6.3 g | 1.3 g | 26 g |
| Carbohydrates | 0.4 g | 27 g | 0 g |

We can format this data into a matrix $A$.

The daily recommended values can be formatted into a matrix $b$.

A feasible point satisfies $Ax \geq b$.

# Diet Problem: Solving the Diet Problem

Linear programs scale extremely well: we can solve one with thousands of foods in a fraction of a second.

**Interactive demo — open in your browser**

**Diet Optimizer**

Interactive demo that solves a diet problem as a linear program (shown live in the lecture slides).

https://ootks.github.io/co250-lectures/demos/DietOptimizer.html

# Linear Programming Review

Given a linear program

$$\begin{array}{ll} \min & c^\mathsf{T} x \\ \text{such that} & Ax \leq b \end{array}$$

A **feasible point** is a point satisfying the constraints $Ax \leq b$.

An **optimal point** is a feasible point $x$ so that for any other feasible $y$, $c^\mathsf{T} x \leq c^\mathsf{T} y$.

# Linear Programming Review

Given a linear program

$$\begin{array}{ll} \min & x_1 \\ \text{such that} & x_1 \leq 0 \end{array}$$

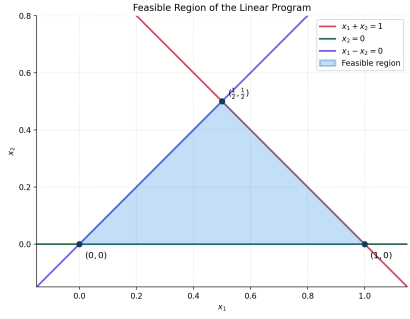
This is feasible, for example $x_1 = 0$ is feasible.

There is no optimal point because the constraints allow $x_1$ to be arbitrarily small.

# Linear Programming Review

The following is an LP

$$\begin{array}{ll} \max & x_2 \\ \text{such that} & x_1 + x_2 \leq 1 \\ & x_1 - x_2 \geq 0 \\ & x_2 \geq 0. \end{array}$$




There is an optimal point for this problem at $(x_1, x_2) = (\frac{1}{2}, \frac{1}{2})$.

# Linear Programming Review

The variables in a linear program are continuous in nature.

Linear programs have a harder time capturing **discrete decisions**.

# Linear Programming Review

Linear programs have a harder time capturing **discrete decisions**.

Note that the solution to the diet problem has fractional values.

If we are trying to decide what to buy at the grocery store, this may be an issue.

# More than Linear Programming: The Cafeteria Problem

A cafeteria offers a number of different options for what to eat.



For each food option, you can either take some or not, but **you can't take part of a food**.

# More than Linear Programming: The Cafeteria Problem

Say that there are $N$ food options.

As in linear programming, we want to have a variable $x_i$ for each $i = 1, \ldots, N$ that represents our decisions.

What should this variable look like?

# More than Linear Programming: Indicator Variables

We want to model **binary decisions:** $x_i$ should take on two values.

An **indicator variable** is a variable that only takes on values of 0 or 1 (and not anything in between).

Indicator variables are very useful for describing binary decisions. For example, we can **count** the number of positive decisions we make by adding up indicator variables.

# More than Linear Programming: The Cafeteria Problem

Say that we have a budget of $B$ and food $i$ costs $c_i$.

We can't spend more than our budget:

$$\sum_{i=1}^{N} c_i x_i \leq B.$$

**Now suppose each food $i$ has a value $v_i$. What should we maximize?**

Our objective is

$$\max \sum_{i=1}^{N} v_i x_i.$$

# More than Linear Programming: The Cafeteria Problem

Overall, our problem becomes

$$\begin{array}{ll} \max & \sum_{i=1}^{N} v_i x_i \\ \text{such that} & \sum_{i=1}^{N} c_i x_i \leq B \\ & x_i \in \{0, 1\} \end{array}$$

This will be our first example of an **integer linear program**.

# Integer Linear Programming

# Integer Linear Programming

An integer linear program (ILP) is an optimization problem of the form

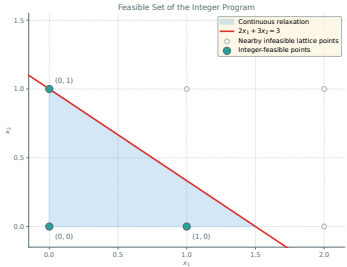
$$\begin{array}{ll} \min & c^\mathsf{T} x \\ \text{such that} & Ax \leq b \\ & x \in \mathbb{Z} \end{array}$$

Here, $x \in \mathbb{Z}$ means that each entry of $x$ is an integer.

More generally, we can let some variables be continuous and others discrete.

# ILP Examples

$$
\begin{array}{ll}
\min & x_1 \\
\text{such that} & 2x_1 + 3x_2 \leq 3 \\
& x_1, x_2 \geq 0 \\
& x \in \mathbb{Z}
\end{array}
$$




**Which points are feasible for the ILP?**

There are three integral points: $(0, 0)$, $(1, 0)$, and $(0, 1)$.

Note that the linear program would have $(1.5, 0)$ as a vertex.

# Integer Linear Programming

Integer programs are much more expressive than linear programs; they can express 'logical constraints'.

We can express that one indicator variable implies another one:

$$x_i \leq y_i.$$

We can express that a constraint may be turned on or off depending on an indicator is true or false.

$$a^\mathsf{T} x \leq b \;\rightarrow\; a^\mathsf{T} x \leq b + M z_i,$$

where $M$ is a large number to make sure that any feasible $x$ will satisfy this is true if $z_i = 1$.

# ILP Examples: The Cafeteria Problem

Note that the Cafeteria problem can be described as an integer program.

$$\begin{array}{ll} \max & \sum_{i=1}^{N} v_i x_i \\ \text{such that} & \sum_{i=1}^{N} c_i x_i \leq B \\ & x_i \in \{0,1\} \end{array}$$

# ILP Examples: The Cafeteria Problem

Note that the Cafeteria problem can be described as an integer program.

$$\begin{array}{ll} \max & \sum_{i=1}^{N} v_i x_i \\ \text{such that} & \sum_{i=1}^{N} c_i x_i \leq B \\ & 0 \leq x_i \leq 1 \\ & x_i \in \mathbb{Z} \end{array}$$

This problem is more often called the knapsack problem.

# ILP Examples: Warehouse Allocation

Food Corp. is looking to decide where to build warehouses to service some grocery locations in the city of Brockton Bay.

There are 3 locations in the Brockton Bay that are available to open a warehouse: North, South, and East.

There are 4 stores in Brockton Bay that need to be serviced, numbered 1 through 4.

Each store should be assigned to one warehouse.

# ILP Examples: Warehouse Allocation

Costs of opening warehouses and transportation costs of going to different stores.

| Location | Cost | Store 1 | Store 2 | Store 3 | Store 4 |
|---|---|---|---|---|---|
| North | 12 | 2 | 4 | 18 | 22 |
| South | 15 | 10 | 7 | 8 | 11 |
| East | 12 | 22 | 17 | 4 | 2 |

# ILP Examples: Warehouse Allocation

**Interactive demo — open in your browser**

**Facility Location Explorer**

Interactive demo for the warehouse allocation (facility location) problem, shown live in the lecture slides.

https://ootks.github.io/co250-lectures/demos/FacilityLocationExplorer.html

# ILP Examples: Warehouse Allocation

What are the decision variables?

Which warehouses do we open? Let $x_i$ be 1 if the warehouse is open and 0 otherwise.

Which warehouses should be connected to which grocery stores? Let $y_{ij}$ be 1 if warehouse $i$ is connected to grocery store $j$.

# ILP Examples: Warehouse Allocation

What are the constraints?

If warehouse $i$ is closed, then no grocery stores connect to it.

For all $i$ and $j$, $y_{ij} \leq x_i$.

Each grocery store should be connected to exactly one warehouse.

For all $j$, $\sum_{i=1}^{3} y_{ij} = 1$.

# ILP Examples: Warehouse Allocation

What is the objective?

Minimize $\sum_{i=1}^{3} c_i x_i + \sum_{i=1}^{3} \sum_{j=1}^{4} d_{ij} y_{ij}$

# ILP Examples: Warehouse Allocation

Assembling

$$
\begin{array}{ll}
\text{min} & \sum_{i=1}^{3} c_i x_i + \sum_{i=1}^{3} \sum_{j=1}^{4} d_{ij} y_{ij} \\
\text{such that} & y_{ij} \leq x_i \\
& \sum_{i=1}^{3} y_{ij} = 1 \\
& 0 \leq x_i \leq 1 \\
& 0 \leq y_{ij} \leq 1 \\
& x_i, y_{ij} \in \mathbb{Z}
\end{array}
$$

# ILP Examples: Cookie Assembly

We have a number of cookies that we need to bake. We want to put all of the cookies on a baking sheet without overlapping.

For simplicity, we will imagine that each cookie is a rectangle (though we can model other shapes easily).

We will also want to discretize each rectangle, so that they are composed of small square 'pixels'.

# ILP Examples: Cookie Assembly

**Interactive demo — open in your browser**

**Cutting Stock Explorer**

Interactive demo accompanying the cookie assembly (packing rectangles onto baking sheets) example, shown live in the lecture slides.

https://ootks.github.io/co250-lectures/demos/CuttingStockExplorer.html

# ILP Examples: Cookie Assembly Setup

Say we are given $K$ cookie sheets, each one of size $\ell \times w$.

We also have $B$ cookies, and cookie $i$ is a rectangle of size $a_i \times b_i$.

**What decisions do we need to make?**

# ILP Examples: Cookie Assembly Decision Variables

First, we decide whether to use each cookie sheet.

Let $y_k$ be a variable that is 1 if cookie sheet $k$ is used, and 0 otherwise, for $k = 1, \ldots, K$.

# ILP Examples: Cookie Assembly Decision Variables

Second, we decide where each cookie goes.

Cookie $i$ can have its top left corner at location $(r, c)$ on sheet $k$ whenever $r \leq \ell - a_i + 1$ and $c \leq w - b_i + 1$.

For each such location, include a variable $x_{ikrc}$ that is 1 if we put cookie $i$ there and 0 otherwise.

# ILP Examples: Cookie Assembly Constraints

We must put each cookie in exactly one location, so for each $i \leq B$,

$$\sum_{k=1}^{K} \sum_{r=1}^{\ell - a_i + 1} \sum_{c=1}^{w - b_i + 1} x_{ikrc} = 1.$$

**What keeps two cookies from overlapping?**

# ILP Examples: Cookie Assembly Constraints

We can only place one cookie in each cell of a sheet that is used.

So for each $k = 1, \ldots, K$, each $r = 1, \ldots, \ell$, and each $c = 1, \ldots, w$,

$$\sum_{i}^{K} \sum_{r'=r-a_i+1}^{r} \sum_{c'=c-b_i+1}^{c} x_{ikr'c'} \leq y_k.$$

Notice how this ties the placement variables back to $y_k$: a cell can only be filled on a sheet we actually use.

# ILP Examples: Cookie Assembly Decision Constraints

The objective here is simple; we want to use as few cookie sheets as possible, which is given by $\sum_{k=1}^{K} y_k$.

Assembling

$$
\begin{array}{lll}
\text{min} & \sum_{k=1}^{K} y_k & \\
\text{such that} & \sum_{k=1}^{K} \sum_{r=1}^{\ell - a_i + 1} \sum_{c=1}^{w - b_i + 1} x_{ikrc} = 1 & \text{for } i = 1, \ldots, B \\
& \sum_{i}^{K} \sum_{r'=r-a_i+1}^{r} \sum_{c'=c-b_i+1}^{c} x_{ikr'c'} \leq y_k & \text{for each } k = 1, \ldots, K \\
& 0 \leq x_{ikrc} \leq 1 & \\
& 0 \leq y_k \leq 1 & \\
& x_{ikrc}, y_k \in \mathbb{Z} &
\end{array}
$$

# Packing vs Covering

Some problems put items into a container. Others cover an object with items.

A **packing** IP is an optimization problem

$$\begin{array}{ll} \max & c^\intercal x_i \\ \text{such that} & Ax \leq b \\ & x \geq 0 \\ & x_i \in \mathbb{Z} \end{array}$$

where $A$ and $c$ have only nonnegative entries.

The upper bounds say that we can only make the entries of $x$ so large before they 'overflow' the packing bound. Our goal is to maximize our value from the packing.

# Packing vs Covering

Some problems put items into a container. Others cover an object with items.

A **covering** IP is an optimization problem

$$\begin{array}{ll} \min & c^T x_i \\ \text{such that} & Ax \geq b \\ & x \geq 0 \\ & x_i \in \mathbb{Z} \end{array}$$

where $A$ and $c$ have only nonnegative entries.

The lower bounds say that we have to make the entries of $x$ large enough to 'cover' the bounds. Our goal is to minimize the cost of the packing.

This is 'dual' to a packing IP.

# Packing vs Covering

**Which of today's examples were packing, and which were covering?**

The knapsack problem is packing: $Ax \leq b$, maximizing. The warehouse and cookie formulations are neither (they minimize, and use equality constraints).

The diet problem is covering: $Ax \geq b$, minimizing.

# Packing vs Covering

ILPs can be neither packing nor covering ILPs.

$$\begin{array}{ll} \min & x_1 + x_2 - x_3 \\ \text{such that} & x_1 + x_2 \geq 2 \\ & x_1 - x_3 \leq 3 \\ & x_1, x_2, x_3 \in \mathbb{Z} \end{array}$$

# Solving ILPs

ILPs are typically much harder to solve than LPs.

LPs with millions of variables and constraints can be solved very quickly.

There are ILPs with hundreds of variables that likely will never be solved exactly.

Building new solvers for ILPs is a large industry.